Table des matières

Labo 01 — Monter la RAW, la compter, et voir ce qu'elle garde 1
Avant de commencer 2
Étape 1 — Monter la couche RAW 2
Étape 2 — Compter la livraison contre ce qu'elle annonce 4
Étape 3 — Voir que tout est du texte 5
Étape 4 — Premier défaut : le même objet, livré plusieurs fois 6
Étape 5 — Deuxième défaut : des codes que personne n'a décodés 8
Étape 6 — Troisième défaut : des textes qui ne sont pas du texte 8
Étape 7 — Créer le dépôt 02_ae 10
Récapitulatif 11
Dépannage 11

Labo 01 — Monter la RAW, la compter, et voir ce qu'elle garde

Labo · Module 1/9 — Formation Analytics Engineering

Ce que vous allez faire. Monter la couche RAW de Boréal dans votre entrepôt, par l'une des deux voies selon que vous avez fait Data Engineering ou non. La compter, table par table, contre le document qui l'annonce. Aller voir de vos yeux les trois formes de défaut qu'elle garde par contrat. Puis créer 02_ae, le dépôt Git dans lequel vivra tout ce que vous écrirez pendant neuf cours. Objectif de fin : 22 tables, 513 857 lignes, zéro écart avec la livraison annoncée, et un dépôt prêt.

Deux terminaux servent dans ce labo, et chaque étape dit lequel. L'Anaconda Prompt, environnement formations actif, sert à Python, à Git et aux commandes du système. La ligne de commande DuckDB, qui s'ouvre par duckdb et se quitte par .exit, sert au SQL.

La liste de contrôle du cours :

- ☐ Socle du guide d'installation en place, mise en situation lue
- ☐ Projet_Boreal/ monté, avec ses sources et son entrepôt
- ☐ Couche RAW en place : 22 tables, 513 857 lignes, 0 écart avec LIVRAISON.md
- ☐ Les trois défauts vus : 26 538 lignes pour 25 483 commandes, KUKLA à 01, AEDAT à 00000000

© 2026 iAlexMG. Tous droits réservés.

- ☐ Un maximum faux constaté sur une colonne de texte : 9 996,26 au lieu de 293 478,95
- ☐ Dépôt Git `02_ae` créé dans `Projet_Boreal`, `.gitignore` en place, `git status` propre

Avant de commencer

Ce cours ne demande que le socle du parcours : Python dans l'environnement conda `formations`, Git, et la ligne de commande DuckDB. Le guide d'installation, `Data_Domain/00_Installation/Guide_installation.md`, les monte une fois pour tout le parcours. Aucun service à allumer : dbt arrive au Cours 03.

Trois vérifications, dans l'Anaconda Prompt :

```
python --version
git --version
duckdb -c "select version();"
```

Vous devez lire une version 3.11 de Python, une version de Git, et `v1.5.5` ou plus récent pour DuckDB. Si l'une des trois manque, le guide d'installation la monte.

Piège — un seul processus à la fois sur l'entrepôt. Un fichier DuckDB ouvert par un processus bloque tous les autres, même en lecture seule. Fermez votre ligne de commande DuckDB par `.exit` avant de lancer un script Python qui écrit dans l'entrepôt, sinon le script échoue sur « le processus ne peut pas accéder au fichier ». Cette règle vaut pour tout le parcours, et elle vous mordra au moins une fois.

Étape 1 — Monter la couche `RAW`

Théorie : `Theorie_01_AE.md`, section 5.

Deux voies mènent au même endroit. Prenez celle qui correspond à votre parcours ; la suite du labo est identique.

Voie A : vous avez fait Data Engineering

Votre `Projet_Boreal/` existe déjà, mais Data Engineering l'a mené plus loin que le point où cette formation commence. Ses sources sont la génération jusqu'à mars 2026, et son entrepôt a chargé les trois mois de 2026 : 580 907 lignes dans `raw`, dont 28 271 en-têtes de commande. Cette formation part de la génération par défaut, qui s'arrête au 31 décembre 2025, et reçoit les mois de 2026 à son Cours 07, un à la fois, comme une équipe d'analytique reçoit ses livraisons. Ramenez donc l'entrepôt au point de départ, en trois gestes. Votre dépôt `01_de` ne bouge pas, et l'entrepôt de Data Engineering reste sur le disque, à côté du neuf.

© 2026 iAlexMG. Tous droits réservés.

D'abord, arrêtez les services de la plateforme. Cette formation n'en allume aucun, et aucun ne doit tenir l'entrepôt ou le lac pendant que le kit les réécrit. Dans Ubuntu, depuis `Projet_Boreal/plateforme` :

```
docker compose stop
```

`stop` arrête les conteneurs sans rien effacer : `docker compose start` les rallume tels quels le jour où vous en aurez besoin. Arrêtés ainsi, ils ne repartent plus d'eux-mêmes au démarrage d'Ubuntu.

Ensuite, mettez de côté l'entrepôt de Data Engineering. Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
move plateforme\entrepot plateforme\entrepot_de
```

La console répond `1 répertoires déplacés.`

Enfin, lancez le kit de la voie B, avec une option de plus. Depuis `Data_Domain` :

```
python _univers/kits/demarrer.py --formation 02 --projet Projet_Boreal --regenerer
```

`--regenerer` autorise le kit à refaire des sources qui ne sont pas celles qu'il attend ; sans elle, il s'arrête avant de toucher à quoi que ce soit. Il regénère la génération par défaut, garde votre `01_de` et le fait tourner, recharge la `RAW` et les séries de l'usine dans un entrepôt neuf, puis compte. Comptez une minute et demie. Son écran de fin doit dire `Conforme`, avec 22 tables et 513 857 lignes dans `raw`, 3 tables et 163 186 lignes dans `argent`.

Pourquoi — un entrepôt neuf, plutôt que de rejouer par-dessus le vôtre. Le chargeur de Data Engineering ajoute les lots qui arrivent et ne retire jamais un lot déjà chargé : c'est ce qui le rend sûr. Relancé sur votre entrepôt avec les sources par défaut, il y laisse les trois mois de 2026, et le kit finit sur `17 écart(s)`, `raw.erp_vbak` à 28 271 lignes au lieu de 26 538. Le lac, lui, n'a pas à être mis de côté : le bronze et l'argent s'y réécrivent d'après les sources, et vos tables Iceberg et Delta du Cours 10 de Data Engineering n'y sont pas touchées.

Voie B : vous n'avez pas fait Data Engineering

Le kit de départ reconstruit ce que cette formation-là aurait livré. Depuis `Data_Domain` :

```
python _univers/kits/demarrer.py --formation 02 --projet Projet_Boreal
```

Le kit crée le dossier, génère les sources de Boréal, y dépose le dépôt de Data Engineering, `01_de`, puis rejoue son chargement et ses séries de l'usine. Comptez une minute et demie, dont l'essentiel est la génération. Son écran de fin affiche l'état de la couche `RAW`, table par table, et le dit conforme quand chaque compte égale ce que Data Engineering annonce : c'est celui que l'étape 2 vérifie.

Le dépôt `01_de` est le code de Data Engineering, pas le vôtre, et vous n'avez pas à le lire. Il est là parce qu'une formation qui reçoit des livraisons finit par en recevoir d'autres : au Cours 07, c'est lui qui chargera les mois de 2026.

Ce que vous devez avoir, dans les deux cas

```
dir Projet_Boreal
```


© 2026 iAlexMG. Tous droits réservés.

Trois dossiers au moins : `sources/` avec les fichiers livrés, `plateforme/` avec l'entrepôt, et `01_de/`, le dépôt de Data Engineering, que vous l'ayez écrit ou que le kit l'ait déposé. L'entrepôt est le fichier `plateforme/entrepot/boreal.duckdb`, et il pèse près de 40 Mo.

Étape 2 — Compter la livraison contre ce qu'elle annonce

Théorie : `Theorie_01_AE.md`, section 5.

C'est le geste du cours, celui que vous referez devant chaque livraison neuve : compter, puis comparer au document qui annonce les comptes.

Ouvrez la ligne de commande DuckDB sur l'entrepôt, depuis `Projet_Boreal` :

```
duckdb plateforme/entrepot/boreal.duckdb
```

D'abord, l'inventaire. Combien de tables, et lesquelles ?

```
select table_name, estimated_size as lignes_estimees
from duckdb_tables()
where schema_name = 'raw'
order by table_name;
```

Vingt-deux lignes sortent. Les noms suivent une convention : `raw.<source>_<fichier>`, ce qui vous dit d'où chaque table vient sans ouvrir le document.

`estimated_size` est une estimation que DuckDB tient pour son planificateur, et qui suffit ici à repérer un ordre de grandeur. Pour un contrôle, il faut un compte exact :

```
select count(*) as tables_raw
from duckdb_tables()
where schema_name = 'raw';
```

Attendu : 22.

Comptez maintenant les cinq tables de l'ERP, celles sur lesquelles la formation travaillera le plus. `union all` empile les cinq comptes en un seul résultat (Cours 07 de la formation SQL avancé, section 6) :

```
select 'erp_kna1' as table_raw, count(*) as lignes from raw.erp_kna1
union all select 'erp_mara', count(*) from raw.erp_mara
union all select 'erp_vbak', count(*) from raw.erp_vbak
union all select 'erp_vbap', count(*) from raw.erp_vbap
union all select 'erp_zserie', count(*) from raw.erp_zserie
order by 1;
```


© 2026 iAlexMG. Tous droits réservés.

Table	Attendu
erp_kna1	7 481
erp_mara	2 459
erp_vbak	26 538
erp_vbap	81 292
erp_zserie	32 597

Ouvrez `01_Data_Engineering/01_Boreal/Solution/LIVRAISON.md`, section 2.2, et comparez. Les comptes doivent tomber exactement. S'ils ne tombent pas, l'étape 1 n'est pas allée à son terme : reprenez-la avant de continuer, parce que tout ce que vous bâtirez ensuite héritera de l'écart.

Le document annonce aussi un total, et vous pouvez le vérifier d'un coup :

```
select sum(estimated_size) as lignes_dans_raw
from duckdb_tables()
where schema_name = 'raw';
```

Attendu : 513 857 lignes sur les 22 tables. Ici l'estimation tombe juste, parce qu'aucune table n'a été modifiée depuis son chargement.

Enfin, le journal des chargements dit combien de fichiers ont été lus pour produire tout cela :

```
select count(*) as fichiers_charges from pipeline.chargements;
```

Attendu : 1 572.

Étape 3 — Voir que tout est du texte

Théorie : `Theorie_01_AE.md`, section 6.

La couche `RAW` garde ce qui a été livré, et un fichier CSV ne transporte aucun type. Vérifiez ce que cela donne :

```
select data_type, count(*) as colonnes
from information_schema.columns
where table_schema = 'raw'
group by 1
order by 2 desc;
```


© 2026 iAlexMG. Tous droits réservés.

Type	Colonnes	Pourquoi
VARCHAR	250	tout ce qui vient d'un fichier, sans exception
TIMESTAMP WITH TIME ZONE	22	la colonne `_charge_le`, une par table, écrite par le chargeur
JSON	2	les réclamations et la télémétrie, livrées en JSON et gardées telles quelles

Regardez maintenant les trois colonnes de lignage sur une table, celles qui disent d'où vient chaque ligne :

```
select _fichier, _lot, _charge_le
from raw.erp_vbak
limit 3;
```

Le chemin est relatif au projet, le lot est le mois de l'extrait, et l'heure est celle de votre chargement. Retenez `_lot` : c'est la colonne qui vous permettra, au Cours 03, de choisir entre deux versions d'une même commande.

Étape 4 — Premier défaut : le même objet, livré plusieurs fois

Théorie : `Theorie_01_AE.md`, section 6.

Comptez les lignes et les commandes distinctes de la table des en-têtes de commande :

```
select
    count(*) as lignes,
    count(distinct vbeln) as commandes,
    count(*) - count(distinct vbeln) as lignes_en_trop
from raw.erp_vbak;
```

Attendu : 26 538 lignes, 25 483 commandes, 1 055 lignes en trop. Ce sont 1 035 commandes qui sont livrées plus d'une fois, soit une sur vingt-cinq. Voyons la répartition. La requête nomme d'abord un résultat intermédiaire par `with` : c'est une expression de table commune, que le Cours 05 de la formation SQL avancé enseigne, sections 2 et 3 :

```
with occurrences as (
    select vbeln, count(*) as n
    from raw.erp_vbak
    group by 1
)
select n as fois_livree, count(*) as commandes
```


© 2026 iAlexMG. Tous droits réservés.

```
from occurrences
group by 1
order by 1;
```

Fois livrée	Commandes
1	24 448
2	1 015
3	20

Deux causes différentes produisent ces doublons, et il faut les distinguer. La première est un incident : l'extrait de mars 2023 a été livré deux fois, sous deux noms de lot.

```
select _lot, count(*) as lignes
from raw.erp_vbak
where _lot like '2023-03%'
group by 1
order by 1;
```

Attendu : 609 lignes au lot `2023-03`, et 609 au lot `2023-03_bis`. Les mêmes commandes, deux fois.

La seconde cause n'est pas un incident, c'est le fonctionnement normal de l'ERP : une commande modifiée revient dans l'extrait du mois de sa modification. Regardez une commande livrée trois fois :

```
select vbeln, _lot, erdat, aedat, netwr
from raw.erp_vbak
where vbeln = '0010006931'
order by _lot;
```

Lot	Créée le	Modifiée le	Valeur nette
2023-02	20230220	00000000	6079.02
2023-03	20230220	20230302	5556.72
2023-03_bis	20230220	20230302	5556.72

Trois lignes pour une commande : celle de février, avant modification ; celle de mars, après ; et sa jumelle du lot livré deux fois. La valeur a changé de 522,30 $ entre les deux versions.

© 2026 iAlexMG. Tous droits réservés.

Voilà pourquoi la `RAW` ne dédoublonne pas. Si elle avait gardé une seule ligne par commande, l'information « au 28 février, cette commande valait 6 079,02 $ » aurait disparu, et personne n'aurait pu la retrouver. Tirer l'état courant, c'est-à-dire la version du lot le plus récent pour chaque clé, sera votre premier geste au Cours 03.

Étape 5 — Deuxième défaut : des codes que personne n'a décodés

Théorie : `Theorie_01_AE.md`, section 6.

Le palier commercial d'un concessionnaire est un code de deux caractères :

```
select kukla as code_palier, count(*) as lignes
from raw.erp_kna1
group by 1
order by 1;
```

Code	Lignes
01	1 127
02	2 566
03	3 788

Trois codes, aucune étiquette. Ce que valent `01`, `02` et `03` n'est écrit nulle part dans la donnée : c'est dans le dictionnaire de l'univers, `_univers/boreal/DICTIONNAIRE.md`, que vous trouverez Or, Argent et Bronze. Ouvrez-le et vérifiez-le maintenant : c'est la seule source de ces correspondances, et un support qui les recopierait finirait par mentir.

La règle vaut pour la formation entière. Un nom de table, un nom de colonne ou un code que vous écrivez dans un modèle sort du dictionnaire, jamais de votre mémoire ni d'un corrigé.

Étape 6 — Troisième défaut : des textes qui ne sont pas du texte

Théorie : `Theorie_01_AE.md`, section 6.

C'est le plus coûteux des trois, parce qu'il ne fait rien planter.

Commencez par la date de dernière modification, une date SAP en texte. `count(*) filter (where …)` ne compte que les lignes qui passent sa condition (Cours 06 de la formation SQL avancé, section 6) :

© 2026 iAlexMG. Tous droits réservés.

```sql
select
    count(*) as lignes,
    count(*) filter (where aedat = '00000000') as jamais_modifiees,
    count(*) filter (where aedat <> '00000000') as modifiees_au_moins_une_fois
from raw.erp_vbak;
```

Attendu : 26 538 lignes, dont 24 230 portent `00000000` et 2 308 une vraie date. La date vide de SAP n'est pas `NULL`, c'est une chaîne de huit zéros, et elle se compare comme n'importe quel texte.

Essayez maintenant de la convertir, des deux façons :

```sql
select
    count(*) filter (where try_cast(aedat as date) is not null) as cast_direct,
    count(*) filter (where try_strptime(aedat, '%Y%m%d') is not null) as avec_le_format
from raw.erp_vbak;
```

Méthode	Dates obtenues
`try_cast(aedat as date)`	0
`try_strptime(aedat, '%Y%m%d')`	2 308

Le cast direct échoue sur les 26 538 lignes, y compris sur les 2 308 qui portent une vraie date : `20230302` n'est pas une date au format que DuckDB reconnaît d'office. Il faut lui donner le format. Avec `try_`, l'échec rend `NULL` au lieu de faire tomber la requête, ce qui est commode et dangereux : une conversion entièrement ratée ressemble à une colonne vide.

Passons au montant. La valeur nette d'une commande est un nombre, rangé en texte :

```sql
select
    min(netwr) as min_texte,
    max(netwr) as max_texte,
    min(netwr::decimal(18, 2)) as min_nombre,
    max(netwr::decimal(18, 2)) as max_nombre
from raw.erp_vbak;
```

	En texte	En nombre
Minimum	0.00	0.00
Maximum	9996.26	293478.95

Le maximum en texte se trompe d'un facteur trente. Rien n'a planté, aucun message n'est apparu, et la requête a rendu un chiffre plausible : un peu moins de dix mille dollars pour une commande de motoneiges, cela ne choque personne. C'est exactement ce qui rend ce défaut coûteux.

© 2026 iAlexMG. Tous droits réservés.

La raison tient à l'ordre alphabétique : `9996.26` commence par un `9`, et aucune chaîne ne le dépasse tant qu'on compare caractère par caractère. `293478.95` commence par un `2`.

À retenir — un type juste n'est pas du confort, c'est de l'exactitude. Le typage de ces colonnes est le travail du `STAGING`, au Cours 03. D'ici là, retenez la règle : tout chiffre tiré d'une colonne de la `RAW` sans conversion explicite est suspect.

Étape 7 — Créer le dépôt `02_ae`

Théorie : `Theorie_01_AE.md`, section 1.

Tout ce que vous écrirez pendant neuf cours vivra dans un dépôt Git à vous, à côté de celui de Data Engineering. Depuis `Projet_Boreal`, dans l'Anaconda Prompt :

```
mkdir 02_ae
cd 02_ae
git init
```

Un projet analytique produit des fichiers qu'on ne versionne jamais : les artefacts de dbt, les journaux, et surtout le fichier de connexion, qui portera un mot de passe dès que vous viserez une plateforme infonuagique. Créez `.gitignore` avec ce contenu :

```
# Artefacts de dbt, rejouables par « dbt build »
target/
dbt_packages/
logs/
.user.yml

# Connexion : jamais dans le dépôt
profiles.yml
.env
```

Puis le premier commit :

```
git add .gitignore
git commit -m "Depot 02_ae : le gitignore"
git status
```

`git status` doit répondre que l'arbre de travail est propre. Si Git signale des fichiers que vous n'attendiez pas, c'est que vous avez lancé la commande depuis le mauvais dossier : vérifiez que vous êtes bien dans `Projet_Boreal/02_ae` et non dans `Projet_Boreal`.

Piège — le dépôt de la formation n'est pas le vôtre. Le dossier `Data_Domain/` que vous avez téléchargé est le dépôt des supports de cours. `Projet_Boreal/` vit à côté et n'y est pas versionné, et `02_ae` est votre dépôt à vous, avec votre historique. Ne commitez jamais votre travail d'apprenant dans le dépôt des formations.

© 2026 iAlexMG. Tous droits réservés.

Récapitulatif

Ce que vous avez maintenant, et que les huit cours suivants supposent :

Ce qui est en place	Comment le vérifier
Les sources de Boréal	`Projet_Boreal/sources/` contient les fichiers livrés
La couche `RAW`	22 tables, 513 857 lignes, 0 écart avec `LIVRAISON.md`
Le dépôt `02_ae`	`git status` propre, `.gitignore` commité

Ce que vous avez vu, et qui commande le Cours 03 :

Le défaut	Sa mesure	Où il se traite
Le même objet livré plusieurs fois	26 538 lignes pour 25 483 commandes	Cours 03, l'état courant
Des codes non décodés	`KUKLA` à `01`, `02`, `03`	Cours 03, le décodage
Des textes qui ne sont pas du texte	un maximum faux d'un facteur trente	Cours 03, le typage

Le Cours 02 ne touche à aucun outil : il conçoit. Avant de transformer quoi que ce soit, il faut savoir vers quoi on transforme, et c'est la modélisation dimensionnelle.

Dépannage

Le message	Ce qu'il veut dire	La parade
`IO Error: Cannot open file ... utilisé par un autre processus`	un autre processus tient l'entrepôt	fermer la ligne de commande DuckDB par `.exit`, ou le script qui tourne
`Catalog Error: Table with name erp_vbak does not exist`	la couche `RAW` n'est pas montée, ou vous avez ouvert le mauvais fichier	reprendre l'étape 1 ; vérifier le chemin de l'entrepôt
Les comptes ne tombent pas juste	le chargement s'est arrêté en route	relancer `python -m pipeline tout` : il ne recharge que ce qui manque

© 2026 iAlexMG. Tous droits réservés.

Le message	Ce qu'il veut dire	La parade
`Le kit s'arrête : les sources présentes sont celles de la génération jusqu'à 2026-03, et vous demandez la génération par défaut.`	voie A : les sources de Data Engineering vont plus loin que ce cours, et le kit refuse de les réécrire sans qu'on le lui dise	relancer le kit avec `--regenerer`
`17 écart(s) avec les attendus de _univers/kits/`, dont `raw.erp_vbak` à 28 271	voie A : le kit a rechargé dans l'entrepôt de Data Engineering, qui garde ses lots de 2026	mettre cet entrepôt de côté, `move plateforme\entrepot plateforme\entrepot_de`, puis relancer la même commande : les sources, déjà refaites, sont gardées
`git init` dans le mauvais dossier	un dépôt Git créé au mauvais endroit	supprimer le dossier `.git` créé par erreur, se replacer, recommencer

© 2026 iAlexMG. Tous droits réservés.